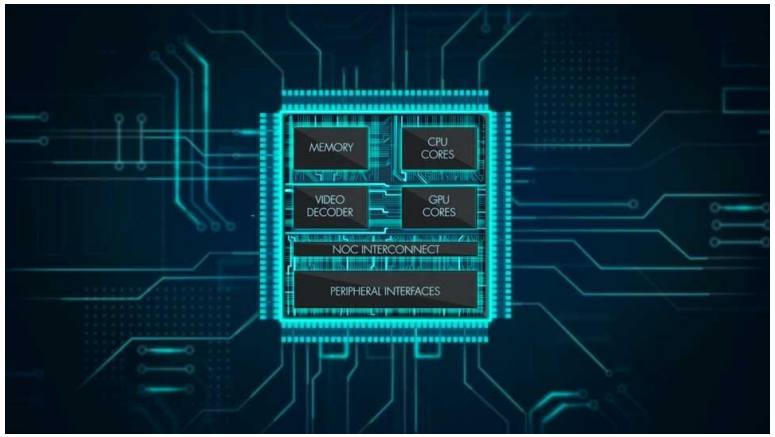
Digital System Design

FSM and Controller



Date : 2026. 04. 30.

Instructor : **Prof. Hyun Kim**

Lecture Note : eclass.seoultech.ac.kr

Office : Mirae Hall #526

Web : idsl.seoultech.ac.kr

Contact : hyunkim@seoultech.ac.kr / 010-9600-5427

Contents

- Why FSM and Controller?
- Finite State Machine(FSM)
- Controller

Why do we need them?

Combinational logic alone is not enough

- Some operations require **ordering and timing**, not just input-to-output mapping.
- We need to decide **what to do** and **when** across multiple cycles.

Typical sequential behavior

- Wait at idle → Receive input → Perform computation → Output result
- These **ordered steps** must be managed systematically.

FSM and Controller

- **FSM** is the basic tool for describing sequential behavior.
- **Controller** applies this FSM to actually **control an entire system**.

Contents

- Why FSM and Controller?
- Finite State Machine(FSM)
- Controller

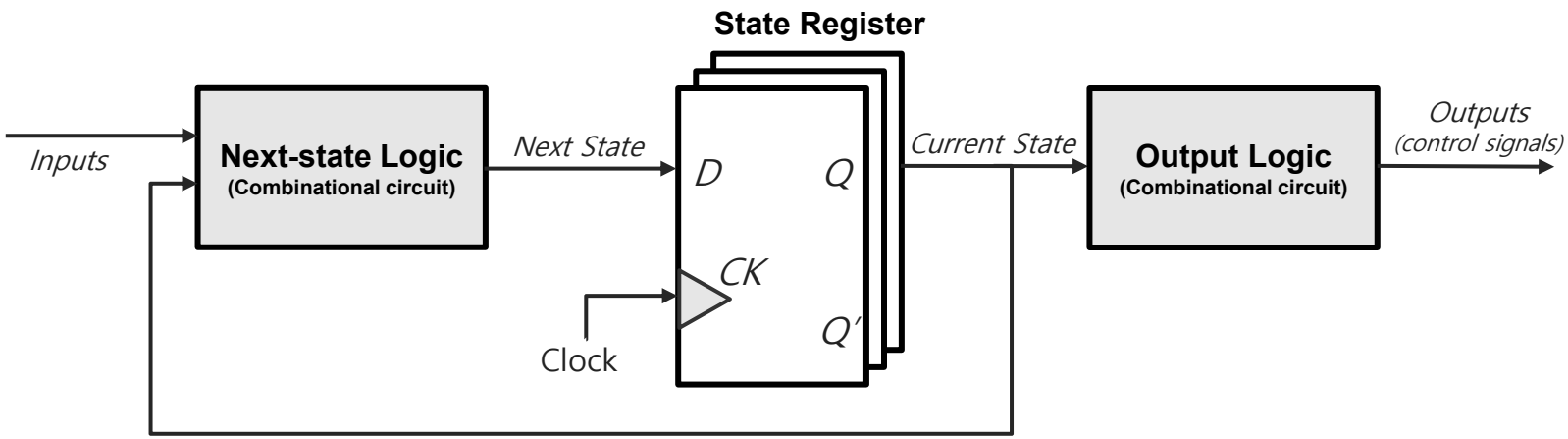
What is FSM?

What is Finite State Machine (FSM)

- A sequential circuit with a **finite number of states**.
- Next state **depends on current state and inputs**; outputs depend on state.

FSM Dataflow

- **Next-State Logic:** Computes next_state from current state & inputs.
- **State Register:** Stores current state, updates on clock edge.
- **Output Logic:** Generates control signals from current state.



<FSM Block Diagram>

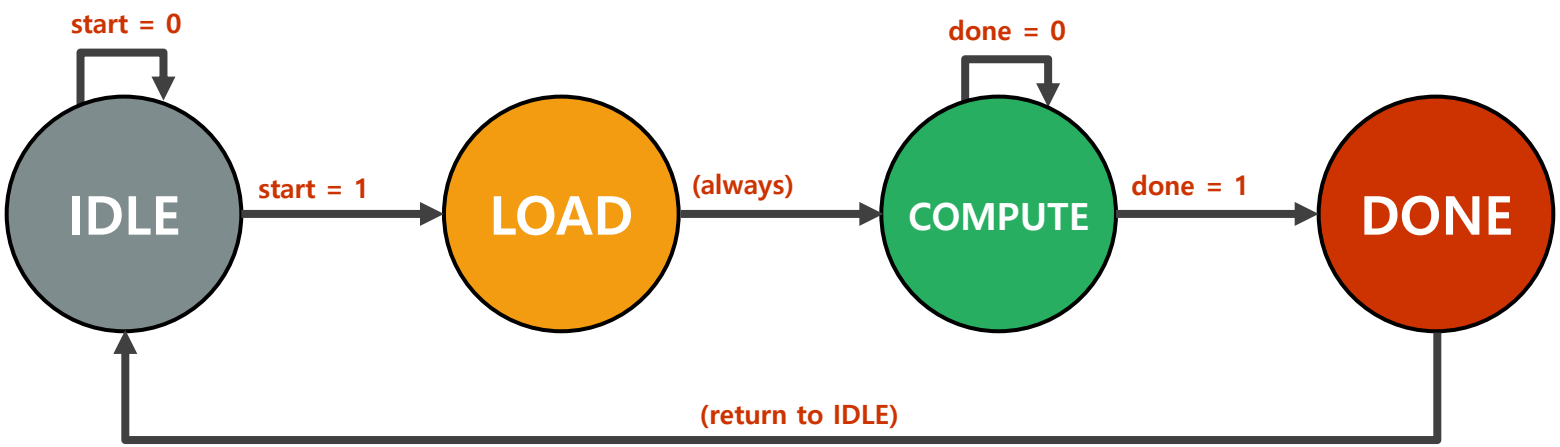
Contents

- Why FSM and Controller?
- Finite State Machine(FSM)
- Controller

A Simple FSM Controller

A minimal 4-state FSM that drives a compute module

- The controller turns each state into the **control signals** that run the datapath.
- IDLE — wait for **start**; datapath idle.
- LOAD — assert **load**; latch inputs and weights.
- COMPUTE — assert **calculate**; run the operation until **done**.
- DONE — assert **output valid** for one cycle, then return to IDLE.

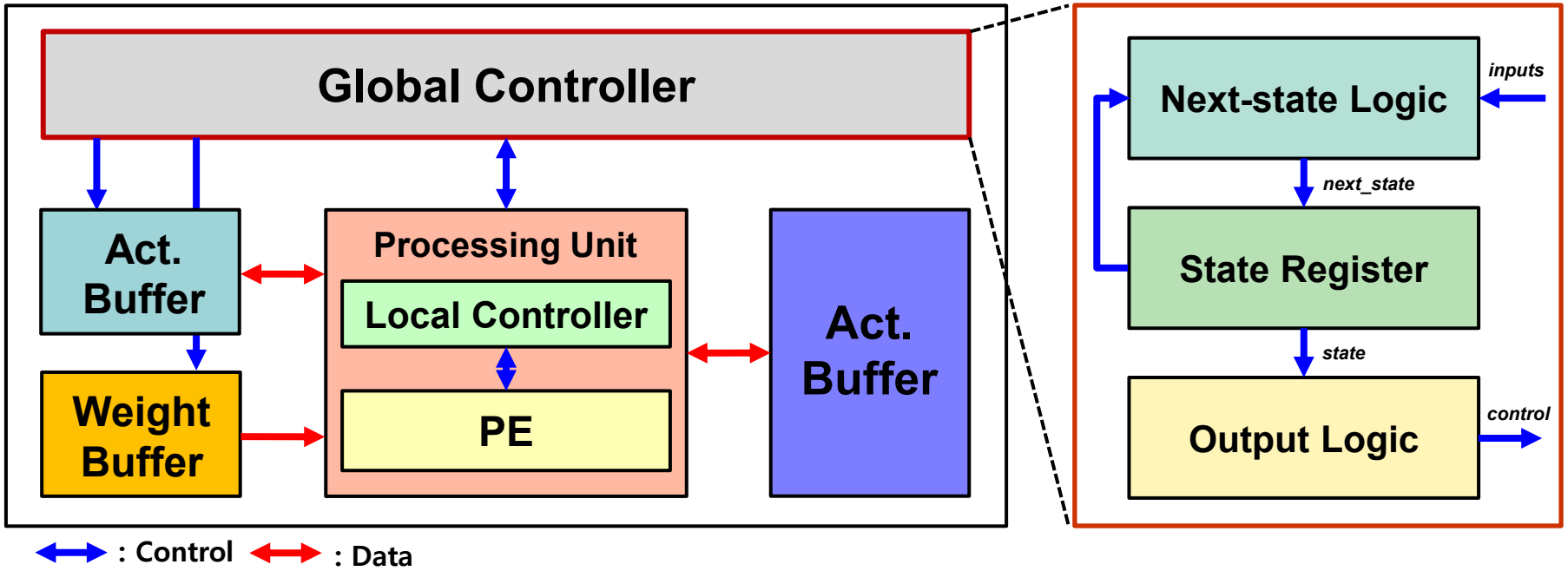


<State Transition Diagram>

Global Controller

Global Controller = Top-level FSM

- Issues **control signals** to Act./Weight buffers and PU.
- Coordinates when data moves and when PU computes each cycle.



<Accelerator Dataflow controlled by Global Controller>

Example in Verilog


```
// Three Part FSM example
parameter IDLE=2'd0, LOAD=2'd1, COMPUTE=2'd2, DONE=2'd3;
reg [1:0] state, next_state;

// state register
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) state <= IDLE;
    else          state <= next_state;
end

// next-state logic
always @(*) begin
    case (state)
        IDLE   : next_state = start ? LOAD : IDLE;
        LOAD   : next_state = COMPUTE;
        COMPUTE: next_state = compute_done ? DONE : COMPUTE;
        DONE   : next_state = IDLE;
        default: next_state = IDLE;
    endcase
end

// output control logic
always @(posedge clk or negedge reset_n) begin
    case (state)
    //~~
    endcase
end
```

Part 1 (State Register)
Updates the *state* signal to *next_state* every clock.

Part 2 (Next-State Logic)
Defines how the FSM transitions between states.

Part 3 (Output Logic)
Generate outputs based on the current states and input.

Thank You!